

JSON

JSON

Lavorare con il formato JSON in Python.

Qual è la funzione di JSON nelle applicazioni

Ogni volta che un'app sul tuo telefono mostra dati dal web — il meteo, i risultati di una ricerca, i post di un social network — quei dati spesso arrivano in formato **JSON**.

JSON è il "linguaggio comune" che i programmi usano per scambiarsi informazioni. Imparare a lavorare con JSON ti permette di fare cose concrete: scaricare dati da internet, salvarli, leggerli.

Cos'è il JSON?

JSON (si legge "djèizon") è un formato di testo per rappresentare dati strutturati. Assomiglia molto ai dizionari e alle liste Python:

```
{
  "nome": "Alice",
  "eta": 16,
  "voti": [8, 7, 9, 6],
  "indirizzo": {
    "citta": "Roma",
    "cap": "00100"
  },
  "attivo": true
}
```

È leggibile dagli esseri umani (se ben indentato) e facilmente interpretabile dai computer.

Importare il modulo

Da Python a JSON: `json.dumps()`

`dumps()` converte un dizionario (o una lista) Python in una **stringa** JSON. Utile per inviare dati o stamparli.

```
dati = {
    "nome": "Alice",
    "eta": 16,
    "voti": [8, 7, 9],
    "attivo": True,      # Python: True → JSON: true
    "media": None        # Python: None → JSON: null
}

# Converti in stringa JSON (su una riga)
json_stringa = json.dumps(dati)
print(json_stringa)
# → {"nome": "Alice", "eta": 16, "voti": [8, 7, 9], "attivo": true,
"media": null}

# Converti in stringa JSON leggibile (con indentazione)
json_bello = json.dumps(dati, indent=4)
print(json_bello)
```

Output indentato:

```
{
  "nome": "Alice",
  "eta": 16,
  "voti": [8, 7, 9],
  "attivo": true,
  "media": null
}
```

Conversione dei tipi: Python ↔ JSON

Python	JSON
<code>dict</code>	<code>{}</code> (oggetto)
<code>list</code>	<code>[]</code> (array)
<code>str</code>	<code>"..."</code> (stringa)
<code>int</code> , <code>float</code>	numero
<code>True</code> / <code>False</code>	<code>true</code> / <code>false</code>
<code>None</code>	<code>null</code>

Da JSON a Python: `json.loads()`

`loads()` fa il contrario: prende una stringa JSON e la converte in un dizionario (o lista) Python.

```
stringa_json = '{"nome": "Alice", "eta": 16, "voti": [8, 7, 9]}'

dati = json.loads(stringa_json)  # Converti la stringa in dizionario
Python

print(dati["nome"])      # Alice
print(dati["voti"])      # [8, 7, 9]
print(type(dati))        # <class 'dict'>
```

Scrivere JSON su file: `json.dump()`

`dump()` (senza la "s") scrive direttamente su un file:

```
studenti = [
    {"nome": "Alice", "eta": 16, "voto": 8},
    {"nome": "Bob", "eta": 15, "voto": 7},
]

with open("studenti.json", "w") as file:
    json.dump(studenti, file, indent=4) # Scrive il JSON nel file, ben
    indentato
```

Il file `studenti.json` conterrà:

```
[
  {
    "nome": "Alice",
    "eta": 16,
    "voto": 8
  },
  ...
]
```

Leggere JSON da file: `json.load()`

`load()` legge un file JSON e lo converte in Python:

```
with open("studenti.json", "r") as file:
    studenti = json.load(file)    # Legge il file e converte in lista Python

for studente in studenti:
    print(f"{studente['nome']}: {studente['voto']}")
# → Alice: 8
# → Bob: 7
```

Gestire errori di formato

Se la stringa JSON non è valida (ad esempio ha una virgola di troppo), Python lancia un errore. Puoi gestirlo con `try/except` :

```
try:
    dati = json.loads("questo non è JSON valido!!!")
except json.JSONDecodeError as e:
    print(f"Errore: il testo non è JSON valido – {e}")
```

Esempio pratico: file di configurazione

Un uso comune di JSON è salvare le impostazioni di un programma in un file che l'utente può modificare:

```
# Salva le impostazioni
config = {
    "tema": "scuro",
    "lingua": "italiano",
    "notifiche": True,
    "volume": 75
}

with open("config.json", "w") as f:
    json.dump(config, f, indent=2)

# La prossima volta che il programma parte, ricarica le impostazioni
with open("config.json", "r") as f:
    config_caricata = json.load(f)

print(f"Tema: {config_caricata['tema']}")      # Tema: scuro
print(f"Lingua: {config_caricata['lingua']}")  # Lingua: italiano
```